

distance.py — teoria i opis modułu estymacji odległości

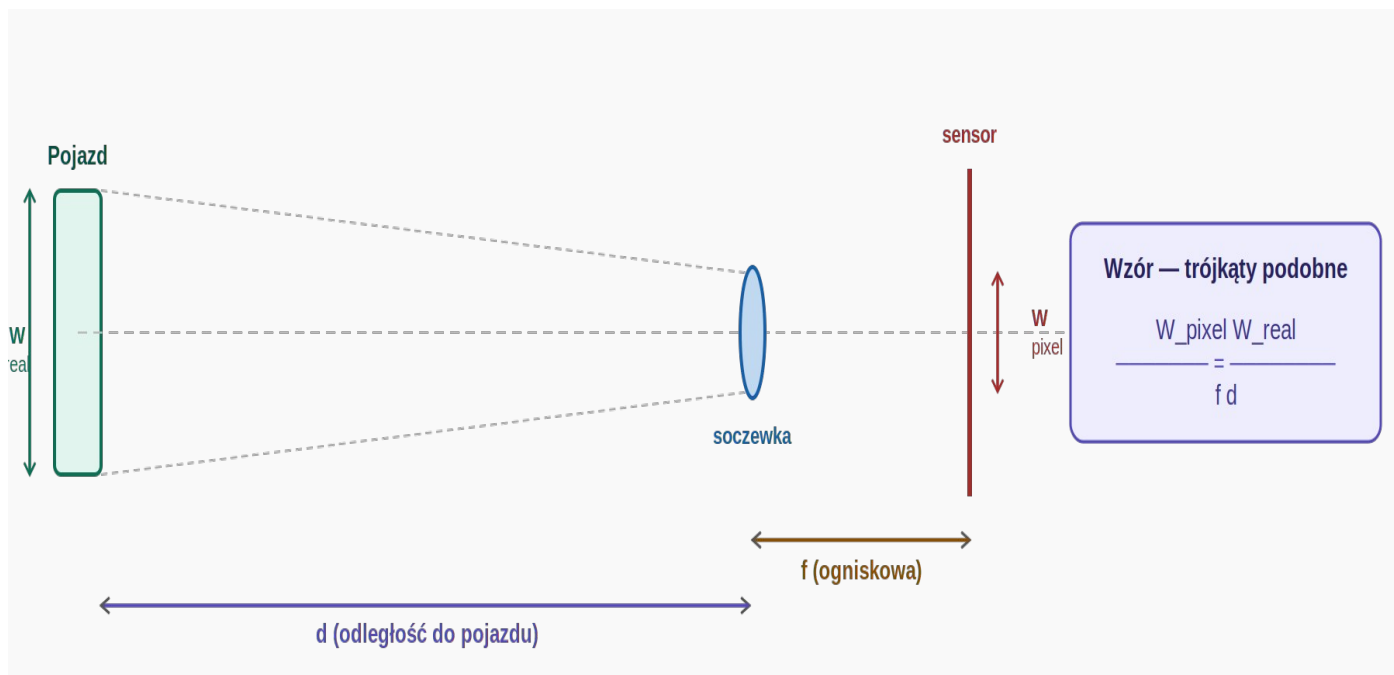
1. Rola modułu w systemie Stop & Go

Moduł distance.py zajmuje się przetworzeniem geometrycznym wyniku detekcji na informację fizyczną. Przyjmuje bbox (x, y, w, h) w pikselach z modułu detector.py i zwraca odległość d w metrach. Ta jedna liczba jest bezpośrednim wejściem maszyny stanów — to ona decyduje czy pojazd hamuje, jedzie czy stoi.

Moduł rozwiązuje trzy osobne problemy: (1) wyznaczenie odległości ze wzoru geometrycznego, (2) kalibrację kamery pozwalającą poznać ogniskową f w pikselach, (3) korekcję zniekształceń soczewki które fałszują pomiar W_pixel. Każdy z tych problemów jest obsługiwany przez osobną metodę klasy.

2. Model kamery otworkowej i wzór na odległość

Podstawą estymacji odległości jest model kamery otworkowej (pinhole camera model). Model zakłada że wszystkie promienie świetlne przechodzą przez jeden punkt — centrum projekcji (soczewkę) — i tworzą obraz na płaszczyźnie sensora. Przy tym założeniu obowiązuje podobieństwo trójkątów między obiektem rzeczywistym a jego rzutem na sensor.



Rysunek 1. Model kamery otworkowej — podobieństwo trójkątów prowadzi bezpośrednio do wzoru na odległość.

2.1 Wyprowadzenie wzoru

Z podobieństwa trójkątów (mały trójkąt przy sensorze i duży trójkąt przy obiekcie):

$$\begin{aligned} W_{pixel} / f &= W_{real} / d \\ \Rightarrow d &= (W_{real} * f) / W_{pixel} \end{aligned}$$

Trzy zmienne po prawej stronie wzoru mają różne statusy. W_{real} to stała fizyczna mierzona raz linijką przed projektem — dla modelu RC jest to szerokość pojazdu RC (np. 0.12 m). Ogniskowa f jest wyznaczana przez kalibrację kamery i zapisywana do pliku. W_{pixel} zmienia się w każdej klatce — jest to szerokość bbox z detector.py.

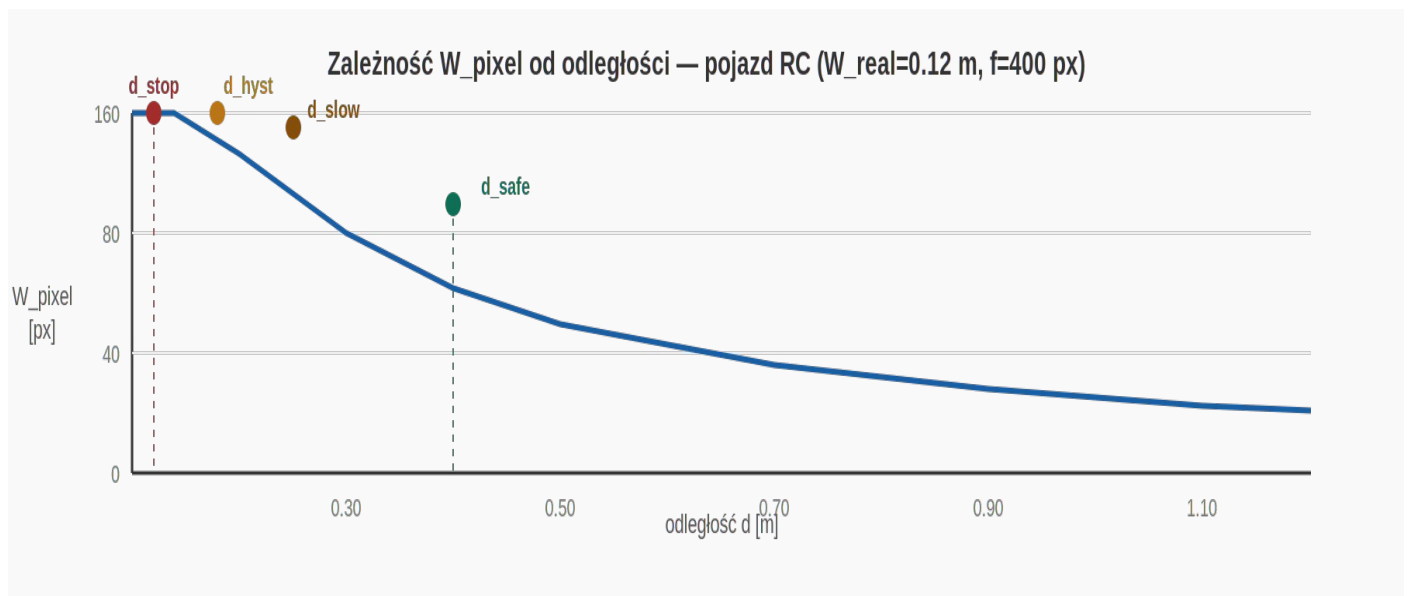
Dlaczego używamy szerokości (w) bbox, nie wysokości (h)?

Szerokość pojazdu jest stabilna — nie zmienia się gdy pojazd jedzie po wzniesieniu lub zjeździe. Wysokość w kadrze zmienia się wraz z pochyleniem drogi i kątem montażu kamery.

Ponadto detektor HOG ma tendencję do lepszego dopasowania poziomego bbox niż pionowego — błąd W_{pixel} jest mniejszy.

2.2 Zależność W_{pixel} od odległości

Wzór $d = W_{\text{real}} * f / W_{\text{pixel}}$ opisuje zależność odwrotnie proporcjonalną. Im bliżej pojazd — tym szerszy bbox w pikselach i tym mniejsze d . Dla modelu RC ($W_{\text{real}} = 0.12 \text{ m}$, $f = 400 \text{ px}$) progi maszyny stanów odpowiadają konkretnym wartościom W_{pixel} :



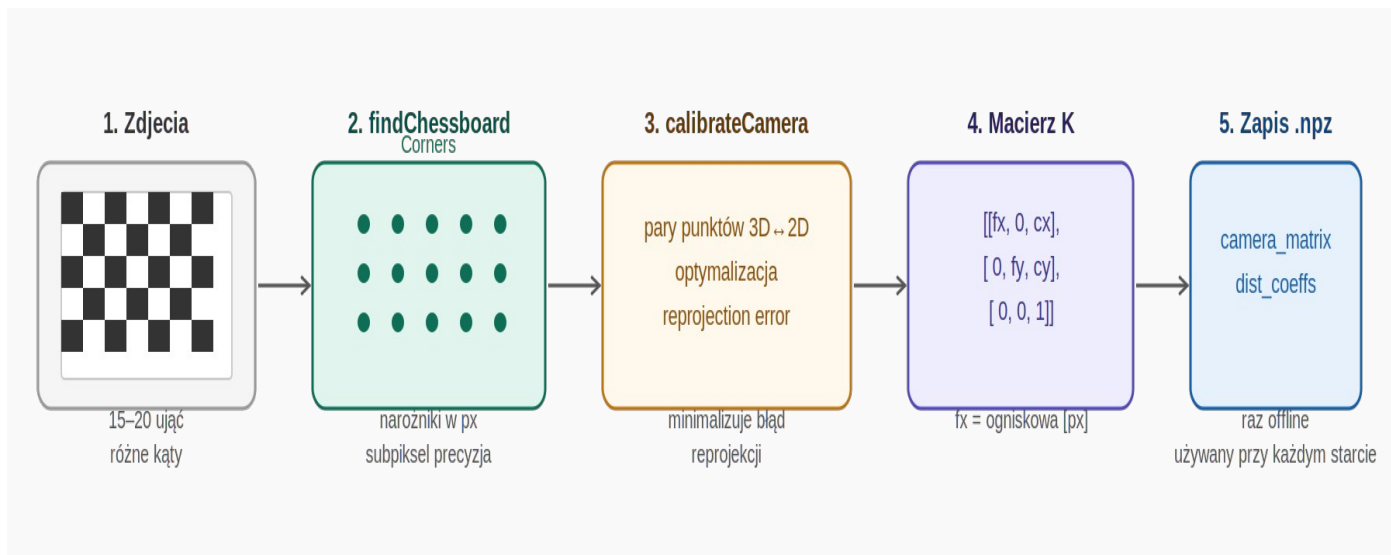
Rysunek 2. Zależność W_{pixel} od odległości dla pojazdu RC. Punkty oznaczają progi maszyny stanów.

$d_{\text{stop}} = 0.12 \text{ m}$	$W_{\text{pixel}} \approx 400 \text{ px}$ — pojazd bardzo blisko, cały bbox wypełnia kadr
$d_{\text{hyst}} = 0.18 \text{ m}$	$W_{\text{pixel}} \approx 267 \text{ px}$ — histereza, ruszamy dopiero przy tej odległości
$d_{\text{slow}} = 0.25 \text{ m}$	$W_{\text{pixel}} \approx 192 \text{ px}$ — zaczynamy zwalniać
$d_{\text{safe}} = 0.40 \text{ m}$	$W_{\text{pixel}} \approx 120 \text{ px}$ — jazda swobodna

3. Kalibracja kamery — skąd bierze się f ?

Ogniskowa f jest podawana przez producentów kamer w milimetrach fizycznych, ale wzór wymaga jej w pikselach. Przeliczenie jest możliwe tylko gdy znamy rozmiar piksela sensora — informację trudną do uzyskania. Zamiast tego wyznaczamy f w pikselach bezpośrednio przez kalibrację na szachownicy.

Procedura kalibracji polega na sfotografowaniu szachownicy o znanych wymiarach z kilkunastu różnych kątów i pozycji. OpenCV porównuje rzeczywiste pozycje narożników (znane, bo wymiary pola szachownicy znamy z linijki) z ich pozycjami na obrazach (mierzone przez `findChessboardCorners`) i wyznacza macierz kamery metodą najmniejszych kwadratów.



Rysunek 3. Pięcioetapowy potok kalibracji kamery. Wynik to macierz K z ogniskowymi f_x , f_y i środkiem optycznym c_x , c_y .

3.1 Macierz kamery K

```
K = [[fx, 0, cx],  
     [ 0, fy, cy],  
     [ 0, 0, 1]]
```

f_x – ogniskowa pozioma [px] ← używana w wzorze
 f_y – ogniskowa pionowa [px]
 c_x , c_y – środek optyczny [px]

Ogniskowe f_x i f_y są teoretycznie równe dla idealnej kamery, ale w praktyce różnią się o kilka procent przez tolerancje produkcyjne sensora. Używamy f_x bo mierzymy szerokość poziomą (W_{pixel}).

Środek optyczny (c_x , c_y) to punkt na sensorze przez który przechodzi oś optyczna. Dla idealnej kamery byłby to dokładny środek obrazu, ale w praktyce jest lekko przesunięty. Kalibracja wyznacza go precyzyjnie — bez tego korekcja zniekształceń działałaby niepoprawnie.

Jak ocenić jakość kalibracji?

`calibrateCamera()` zwraca RMS reprojection error — średni błąd w pikselach między zmierzonymi a reprojektowanymi narożnikami.

Dobra kalibracja: RMS < 1.0 px

Akceptowalna: RMS 1.0 – 2.0 px

Zła (powtórzyć): RMS > 2.0 px

Zły wynik najczęściej oznacza: za mało zdjęć, niewystarczające pokrycie kątowe, lub rozmytą szachownicę (wstrząsy kamery).

3.2 Współczynniki zniekształceń dist_coeffs

Poza macierzą K kalibracja wyznacza wektor pięciu współczynników opisujących dystorsję soczewki:

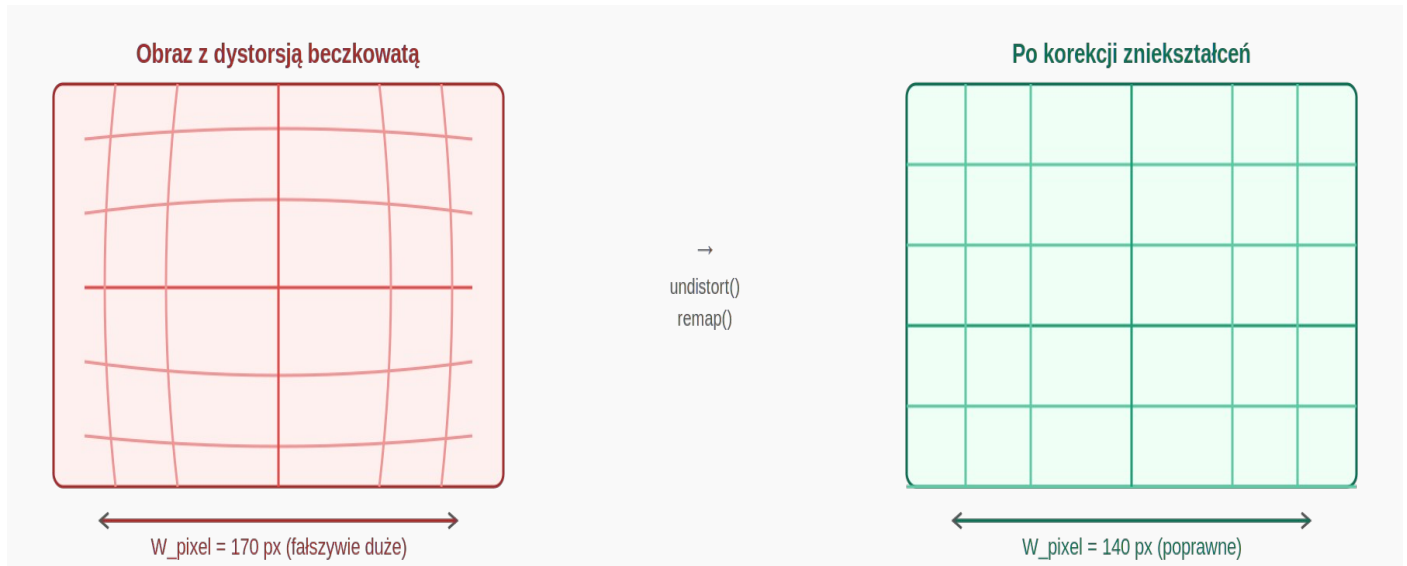
```
dist_coeffs = [k1, k2, p1, p2, k3]
```

k_1 , k_2 , k_3 – dystorsja radialna (beczkowate lub poduszkowe zniekształcenie)
 p_1 , p_2 – dystorsja styczna (nierówne ustawienie soczewki względem sensora)

Tanie kamery USB i moduły kamer do SBC mają znaczną dystorsję radialną. Szerokokątne kamery (np. rybie oko) mają k_1 , k_2 bardzo duże. Bez korekcji W_{pixel} jest fałszywie duże przy krawędziach kadru, co przekłada się na zaniżoną estymację odległości.

4. Korekcja zniekształceń soczewki

Metoda `undistort()` stosuje korekcję zniekształceń na każdej klatce przed detekcją. Dzięki temu `detector.py` widzi geometrycznie poprawny obraz, a `W_pixel` mierzone z `bbox` jest wiarygodne.



Rysunek 4. Dystorsja beczkowa (typowa dla tanich kamer) fałszuje `W_pixel`. Po korekcji siatka jest prosta, pomiar poprawny.

4.1 Mapy korekcji — dlaczego `initUndistortRectifyMap` zamiast `undistort`?

OpenCV oferuje dwa podejścia do korekcji zniekształceń. Funkcja `cv2.undistort()` liczy korekcję na żądanie dla każdej klatki od nowa — prosta w użyciu, ale kosztowna obliczeniowo przy 30 fps. Podejście z mapami jest szybsze:

```
# Raz (przy starcie):
map1, map2 = cv2.initUndistortRectifyMap(
    camera_matrix, dist_coeffs, None,
    camera_matrix, (width, height), cv2.CV_16SC2
)

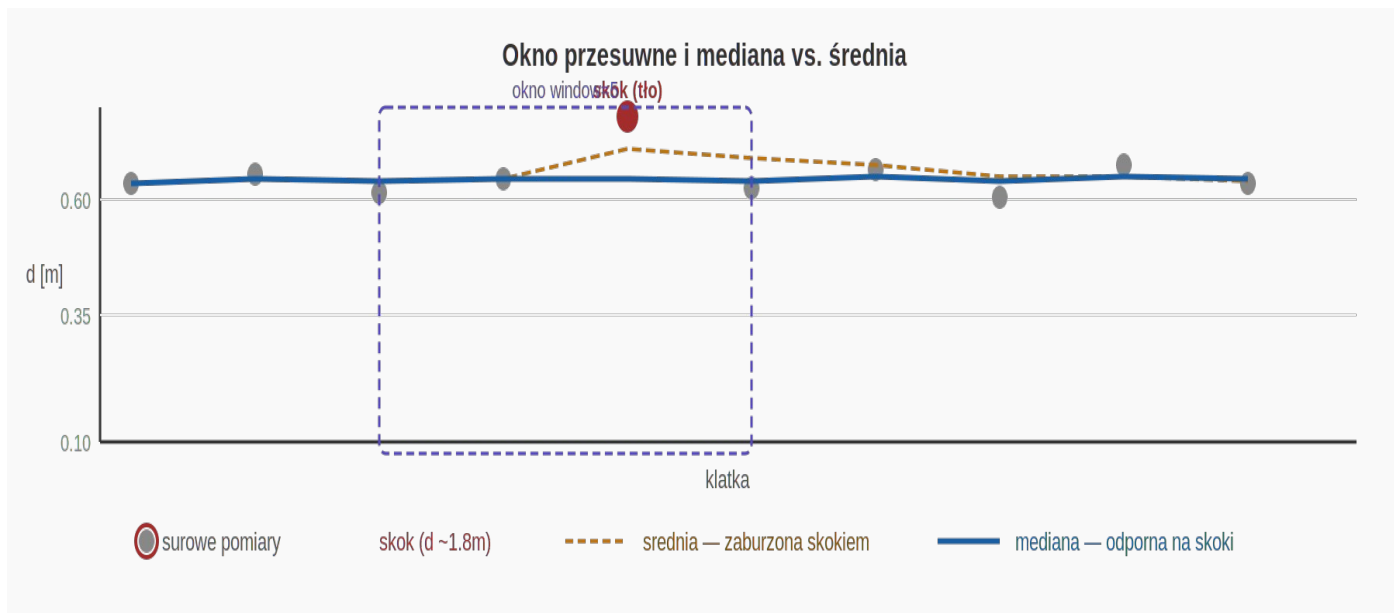
# Co klatka (30× na sekundę):
corrected = cv2.remap(frame, map1, map2, cv2.INTER_LINEAR)
```

`initUndistortRectifyMap` oblicza raz dla każdego piksela wyjściowego: skąd w obrazie wejściowym należy pobrać wartość. Wynikiem są dwie macierze `map1` i `map2` (x i y osobno). `remap()` stosuje te preobliczone mapy — to tylko jedno przepisanie pikseli, bez żadnych ciężkich obliczeń geometrycznych.

CV_16SC2	Format map — 16-bitowe inty ze znakiem, 2 kanały (x, y). Szybszy niż <code>CV_32FC2</code> (floaty) bo CPU wykonuje operacje całkowitoliczbowe szybciej.
INTER_LINEAR	Interpolacja biliniowa przy remap. Gdy pozycja źródłowa wypada między pikselami, wartość jest interpolowana z 4 sąsiadów. Lepiej niż <code>INTER_NEAREST</code> , szybciej niż <code>INTER_CUBIC</code> .

5. Wygładzanie pomiaru — mediana w oknie przesuwным

Surowy pomiar `d` obliczony z pojedynczej klatki jest podatny na szum. Tracker może przez jedną klatkę złapać tło zamiast pojazdu, dając absurdalnie duże `W_pixel` (np. 600 px zamiast typowych 80 px). Bez filtrowania taki pojedynczy skok mógłby wywołać fałszywe ruszenie lub hamowanie.



Rysunek 5. Mediana z okna 5 klatek jest odporna na pojedyncze skoki. Średnia arytmetyczna ulega zakłóceniu, mediana nie.

5.1 Mediana vs. średnia arytmetyczna

```
Historia pomiarów (window=5): [0.58, 0.60, 1.80, 0.61, 0.59]
                                ^^^^ skok

Średnia: (0.58+0.60+1.80+0.61+0.59) / 5 = 0.836 m ← zaburzona
Mediana: sort → [0.58, 0.59, 0.60, 0.61, 1.80]
          środkowy element = 0.60 m ← poprawna
```

Mediana jest statystyką odporną (robust statistic) — na zbiór n elementów, do 50% wartości odstających nie zmienia jej wartości. Dla okna 5 klatek jeden skok jest ignorowany całkowicie. Dla okna 7 klatek ignorowane są dwa skoki. Okno 5 klatek to dobry punkt startowy — daje ~167 ms wygładzenia przy 30 fps, co jest wystarczające dla dynamiki ruchu modelu RC.

Okno przesuwne — implementacja przez listę:

```
history.append(d)    # dodaj nowy pomiar na końcu
if len(history) > window:
    history.pop(0)    # usuń najstarszy z początku
```

Dla małych okien (5–10) list jest wystarczający.
Dla dużych okien lepiej użyć `collections.deque(maxlen=window)`
— `pop(0)` na liście ma złożoność $O(n)$, `deque.popleft()` $O(1)$.

5.2 Zachowanie przy braku pomiaru

Gdy `bbox = None` (detektor nie widzi pojazdu), metoda `estimate_with_smoothing()` nie dodaje niczego do historii. Przy następnym wywołaniu metoda zwraca medianę z poprzednich poprawnych pomiarów — co jest lepszym zachowaniem niż nagłe `None` które wywołałoby panikowe hamowanie.

Historia jest przekazywana z zewnątrz (z `main.py`) jako lista — nie jest przechowywana wewnątrz klasy. To celowy wybór projektowy: `DistanceEstimator` pozostaje bezstanowy względem historii pomiarów, co ułatwia testowanie i ewentualne użycie wielu instancji.

6. Interfejs publiczny

`init_undistort_maps(w, h)`

Oblicza mapy korekcji zniekształceń. Wywołać raz po stworzeniu obiektu, przed

	pierwszą klatką.
<code>undistort(frame)</code>	Koryguje zniekształcenia soczewki na klatce ndarray. Zwraca poprawioną klatkę. Wywołać przed <code>detector.update()</code> .
<code>estimate(bbox)</code>	Oblicza d z wzoru $d=W_real*f/W_pixel$. Zwraca float [m] lub None. Bez wygładzania.
<code>estimate_with_smoothing(...)</code>	Wersja z medianą w oknie przesuwным. Zalecana w pętli głównej. history to lista modyfikowana in-place.
<code>focal_length</code>	Property. Zwraca f_x [px] z macierzy kamery. Przydatne do debugowania i logowania.

7. Źródła błędów i ich wpływ

Błąd <code>W_real</code>	Zły pomiar rzeczywistej szerokości pojazdu. Błąd 10% w <code>W_real</code> = błąd 10% w d . Mierzyć linijką po złożeniu pojazdu RC.
Błąd <code>W_pixel</code>	Niedokładny <code>bbox</code> z detektora — najczęstsze źródło błędów. Redukowany przez kalibrację kamery i medianę.
Błąd kalibracji	Zła kalibracja = zła ogniskowa f_x . $RMS > 2px$ znacząco zaburza wyniki. Powtórzyć kalibrację z więcej zdjęciami.
Dystorsja	Nieskorygowana dystorsja fałszuje <code>W_pixel</code> przy krawędziach kadru. Dlatego korekcja <code>undistort()</code> jest obowiązkowa.
Pochylenie drogi	Na wzniesieniu pojazd jest niżej w kadrze, <code>W_pixel</code> może być inne. W modelu RC (płaskie podłoże testowe) pomijalne.

8. Odniesienia literaturowe

1. Kaehler A., Bradski G. — Learning OpenCV 3: Computer Vision in C++ with the OpenCV Library, O'Reilly Media, 2016.

Rozdział 18: kalibracja kamery, model otworkowy, `calibrateCamera`, `initUndistortRectifyMap`.

Rozdział 2: struktury danych obrazu, ndarray, przestrzenie barw.

2. Zhang Z. — A Flexible New Technique for Camera Calibration. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2000.

Oryginalna metoda kalibracji na szachownicy zaimplementowana w OpenCV.

3. Dokumentacja wybranego komputera jednopłytkowego (Raspberry Pi / Jetson Nano).

Specyfikacja modułu kamery CSI/USB: rozmiar sensora, kąt widzenia, zniekształcenia.